

ARSENAL

black hat

Bastet

An Infrastructure for Benchmarking LLM
Smart Contract Auditing



SunSec

CISO
XREX Inc.



Alice Hsu

Sr. Security Researcher
OneSavie Labs

Agenda

- Background
 - About Bastet
- Competition for LLM Identification of Smart Contract Vulnerabilities
- Outstanding Participants' Solutions
 - Demo
- Conclusion
 - Take Away

Background

How Bastet Started

Technical Need

- Static analyzers rely on AST / Regex rules
- Cannot understand execution logic
 - Frequent logic flaws in DeFi go undetected

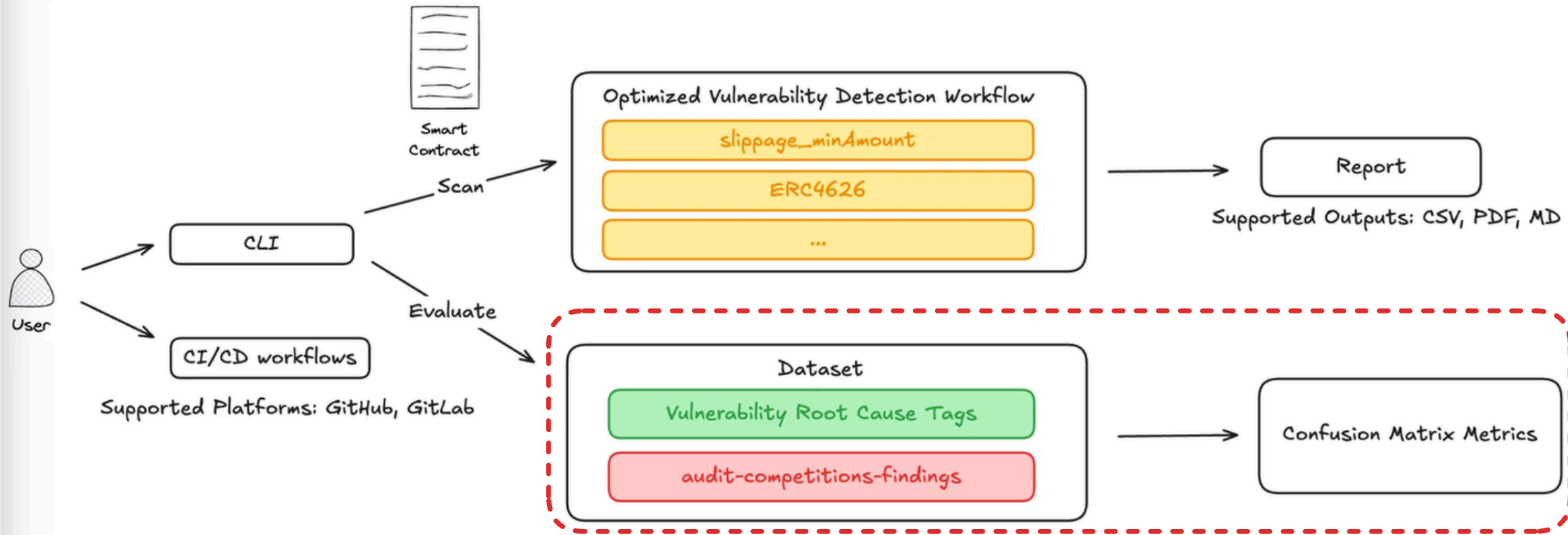
Barriers from Closed AI Security Tools

- Most AI Audit Tools are Closed Source
- Impossible to reproduce or verify their claimed performance
- Researchers and developers can't build on each other's work

About Bastet



Overview



Bridging Web3 Security and AI Talent

OneSavie Bastet

Competition for LLM Identification of Smart Contract Vulnerabilities



- Overview
- Data
- Code
- Models
- Discussion
- Leaderboard
- Rules
- Team
- Submissions
- Settings

Overview

Objective

Build an automated analysis process that can accurately recognize and output the complete Tag, Subtag, and description, in order to improve the efficiency and accuracy of vulnerability classification.

Please use the Tag Definition provided in the Bastet project to identify vulnerabilities in the competition dataset's Repos.

For each detected vulnerability, please submit the following information:

Competition Host

BastetBlackCat



Prizes & Awards

Kudos [Edit](#)

Does not award Points or Medals

Participation

302 Entrants

207 Participants

194 Teams

6,157 Submissions

Competition for LLM Identification of Smart Contract Vulnerabilities

The Competition at a Glance

One of the largest open benchmarks built to date for evaluating LLMs on real-world smart contract vulnerabilities.

330

Entrants

221

Teams

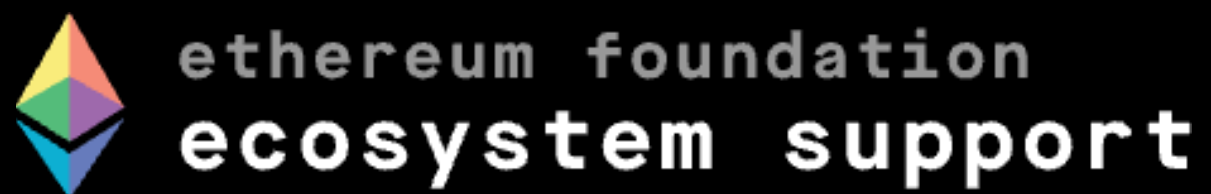
7,565

Submissions

3 Months

Apr 1 – Jul 1, 2026

BACKED BY



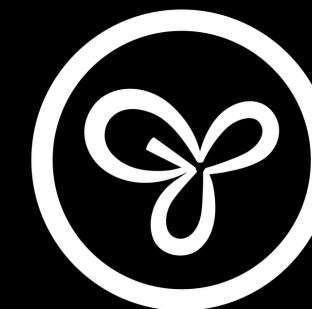
Ethereum Foundation

Ecosystem Support Program



IEEE CIS

Singapore Chapter



GCC

Global Chinese Community of Universal Digital Commons

About the Competition

The Dataset

Co-labeled with DefiHackLabs, a leading Web3 security community. Real vulnerabilities from live smart contract repositories, tagged by experienced researchers.

The Task

Identify the Tag, Subtag, Severity, and Description for every vulnerability, classified using Bastet's own Tag Definition.

The Score

Rewards precise, well-explained findings and penalizes over-reporting, because **precision is enforced by design**. An LLM that floods a repo with guesses to fish for hits **scores worse, not better**, the same way real auditors pay a real cost for raising false alarms.

Outstanding Participants' Solutions

Meet the Winner: Specters

An 8-stage multi-agent pipeline built on Cloudflare's open Project Glasswing research.

688

BEST PUBLIC SCORE

The Engine: GPT-5.5 hunts, Sonnet 4.6 judges. Pure HTTP, no local weights, no GPU on stage.

Model-Agnostic: Swaps in open-source Gemma 4 or Qwen over the same interface.

The Spine: 8 Cloudflare research stages, plus Specter's own extensions.

System Overview: Two Layers

Layer 1: Specter Engine ★ Walkthrough

Hunt: GPT-5.5. Judge: Sonnet 4.6. Two frontier models talk over plain HTTP.

No local weights, no GPU on stage.

Model-Agnostic: same interface swaps in open-source Gemma 4 or Qwen.

Layer 2: Post-Processing

Dedupe: collapses duplicate root causes.

Reachability: drops what can't actually be triggered.

PoC Verification: confirms the exploit on-chain.

Description Refinement: produces the final submission format.

Recall comes from Layer 1 casting wide. Precision comes from Layer 2 converging it down.

The 8-Stage Pipeline



Gapfill and Feedback both loop back into Hunt for one more pass before the pipeline moves on.

8-stage spine: **Cloudflare's open Project Glasswing** methodology, extended with Specter's own sub-stages.

Specter Analysis Pipeline

01 Recon

Project profiling

Graph extraction

Attack-surface model

Specialist routing

02 Hunt

Multi-agent discovery

KB-guided specialists

Taxonomy-aware analysis

Post-processing

Recall optimization

03 Validate

Tag normalization

Adversarial review

Structural FP filter

Graph-based verification

04 Gapfill

Coverage analysis

Targeted re-Hunt

05 Dedupe

FoT consolidation

Inheritance dedupe

Same-location merge

Cross-location root-cause

Duplicate suppression

06 Reachability

Reachability analysis

Entry-point mapping

Exists-vs-reachable

In-scope gating

07 Feedback

Finding feedback

Iterative refinement

Confirmed → re-Hunt loop

08 Report

Structured reporting

Description refinement

Two Design Lessons

Adversarial Review

A second model, prompted like a defense attorney, re-reviews every finding and can only keep or drop, never add. Making the model and the prompt disagree with the hunter is exactly what catches false positives.

Split the Chain

“Bug exists” and “bug is reachable” are judged as two separate questions, and internal-only exploits get dropped or downgraded. A bug in an internal function that nothing external can call is not an exploitable vulnerability.

High Recall, Then Converge

Recall without convergence is just noise. Convergence without recall misses real bugs.

Cast Wide

The scanner deliberately over-produces candidates.
Better to over-collect than miss a real bug.

Converge

PoC verification, dedupe, reachability, and description refinement collapse it into a clean list.

The public leaderboard traced exactly that arc:

260.80

Cast Wide, Early Build

409.3

Converging: Post-Processing Added

688

Fully Converged: Public Best

The Proof Layer

Most LLM auditors stop at a description. This one ships a working exploit.

Beyond Glasswing's spine: what Specter adds after a finding is confirmed.

PoC Verification

Compiles a Foundry PoC that reproduces the finding on-chain.

Invariant Testing

Forge-fuzzes protocol invariants to confirm the break.

Solvency & Bad-Debt

Detects insolvency and bad-debt states directly.

Breaking Call-Sequence

Reports the exact transaction sequence that triggers it.

Bonus: Scan Less, Miss Nothing

Vulnerabilities aren't spread evenly across files.

A small, trained retriever ranks the codebase first, so the expensive LLM only reads the suspicious ones.



42

In-Scope Files

8

Actually Read (19%)

80%

Fewer Tokens

KEY INSIGHT

recall@3 stays near 1.0, so nothing at-risk falls out of scope.

Qwen2.5-Coder retriever, SCAR plus BM25 hybrid ranking.

Demo

Live Demo: Recon



128 files scanned.

Before hunting begins, the pipeline builds a call graph and models 41 entry points, the real attack surface.

Stage 1 of 8. The pipeline narrows scope before any code is read.

Live Demo: Validate

FILES 128

SPECIALISTS 6

FOUND 37

KEPT 21

REPORTED 0

37 candidates found.

Recon

Hunt

3 **Validate** LLM

4 Gapfill

5 Dedupe

6 Trace

7 Feedback

8 Report

```
glasswing - stage 03 · Validate
```

```
► adversarial judge (cross-model) reviewing 37  
  defense-attorney prompt · keep or drop only  
x dropped 16 unprovable / structural noise  
✓ 8 deterministic precision layers applied  
✓ 21 findings survive
```

37 candidates found.

A second model reviews each one, prompted like a defense attorney. It can only keep or drop, never add.

16 are dropped as false positives. 21 survive.

Stage 3 of 8. This is the step that keeps precision high.

Live Demo: Report

FILES

128

SPECIALISTS

6

FOUND

38

KEPT

19

REPORTED

19

✓ Recon

✓ Hunt

✓ Validate

✓ Gapfill

✓ Dedupe

✓ Trace

✓ Feedback

8

Report

DET

glasswing - stage 08 - Report

▶ emitting structured JSON - fixed schema

tag - subtag - severity - location - PoC

✓ schema validated

✓ glasswing_reports/<repo>.report.json

128 files in.

19 findings out.

Every one schema-validated: tag, subtag, severity, location, and a working proof of concept.

Stage 8 of 8. Cast wide, then converge. That's the whole idea.

Conclusion

A Benchmark Can Only Test What We Already Know

Known vs. Unknown

A benchmark measures how good your tool is. It can't tell you how it performs on a contract nobody's looked at yet.

No Adversarial Pressure

A static dataset doesn't fight back. Live contracts face attackers who adapt in real time.

Different Stakes

A false positive costs points here. In production, a false negative can cost millions.

The Tool Works Both Ways

DEFENDERS

Faster Discovery

What took days of manual review, LLMs scan in minutes.

ATTACKERS

Exploit Factories

Once a bug is found, LLMs help generate the PoC. Weeks collapse into hours.

What We Built for This Ecosystem



The Dataset

Real Vulnerabilities Co-labeled with DefiHackLabs, built specifically for Web3.



The Benchmark

Fair and Reproducible

Structured, provable evaluation for LLM auditing, not a popularity contest.



The Bridge

234 Participants, One Challenge

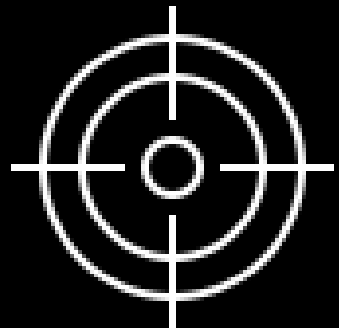
AI researchers and Web3 security talent, working the same problem.

Take Away



Speed & Scale, Rewritten

Vulnerability discovery collapsed from days of manual review to minutes and now generates verifiable proof-of-concept exploits automatically.



Precision Is Designed, Not Luck

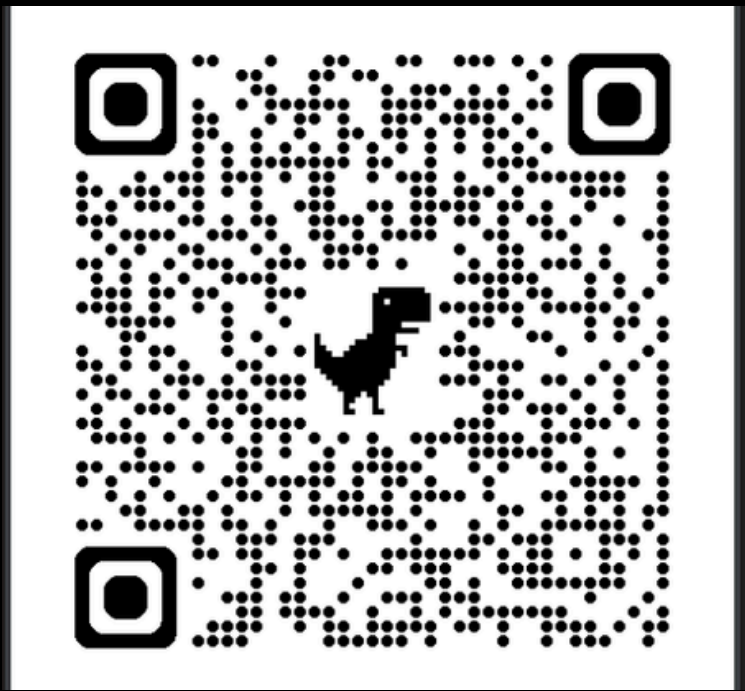
Adversarial review, reachability analysis, and exploit verification converge findings down not a bigger model.



Passing the Benchmark Isn't Battle-Ready

Production faces unknown vulnerabilities, adversarial attackers, and asymmetric stakes the same capability cuts both ways.

Join & Contribute



Bastet Github Repo

